

ESP Failure Prediction

Team Guide: the whole project, explained in plain English

Samsung Innovation Campus · AI Capstone

Group-3 · Data Drillers

Muhammed Elfeelali · Anas Eddanfor · Moataz Fakah · Abdullah Shawesh · Dana Auzi · Babiker

Yousuf Ahmed Hassan

29 September 2026

Who this guide is for

This guide is for us, the team. The formal report is written for the instructor and is short on purpose. This one takes its time. It explains every part of the project in simple words, with examples, so that any one of us can explain any part of it during the presentation, even the parts someone else built.

How to read it

- **Parts 1 to 3** tell the story: the problem, what we built, and where the data comes from.
- **Parts 4 to 6** explain how the models work and how we measure them.
- **Parts 7 to 9** give the results, the experiments that got us there, and the frequency work.
- **Part 10** explains the live dashboard and the demo video.
- **Part 11** is presentation prep: slide by slide, plus the questions we are most likely to get.
- **Parts 12 and 13** cover the project folder, how to run everything, and the limitations.
- **The glossary** at the end explains every technical word used here.

Numbers. Every number in this guide comes from the saved result files in 3_results/ or from the live dashboard run on well 17. If you quote a number in the presentation, quote it from here.

Part 1 · The problem

1.1 What an ESP is

An **ESP (Electrical Submersible Pump)** is a long electric pump that sits at the bottom of an oil well and pushes the oil up to the surface. It is one of the most common ways to lift oil when the reservoir pressure alone is not enough. A single ESP string has four main parts, from bottom to top:

Part	What it does	Why it matters to us
Motor	Electric motor that turns the pump shaft	Has a minimum and maximum safe speed (Hz)
Protector	Seals the motor from well fluid and carries the pump's thrust	Its thrust bearing limits the maximum speed
Gas separator	Removes free gas before the fluid enters the pump	A weak separator makes gas lock more likely
Pump	Many stacked stages (impellers) that lift the fluid	Has its own safe speed range and a best-efficiency speed (BEP)

At the surface, a **variable speed drive** sets how fast the pump turns. The speed is given as a **frequency in Hz**. Around 50 to 60 Hz is typical. Faster means more fluid, but also more strain.

1.2 What a trip is and why it hurts

A **trip** (or shutdown) is when the pump's protection switches it off to save it from damage. From that moment the well produces nothing. It can take hours to restart, and on a remote well it can take a day or more for a crew to arrive. Every restart also wears the equipment, and a pump that fails completely needs a **workover**, which means pulling the whole string out of the well. That is very expensive.

1.3 Why a trip can be predicted

Today, field alarms work on fixed thresholds: if a sensor crosses a line, the alarm fires. That line is usually crossed at the moment of failure or just before it, so the engineer learns about the trip after it happens.

But most trips do not come from nowhere. Gas builds up at the intake over hours. Scale narrows the pipe slowly. The motor gets a little hotter every day. The well's fluid level drops bit by bit. These changes show up in the sensor data long before the trip. **Those hours are the whole opportunity.** Our project tries to use them.

1.4 The four failure causes we predict

Cause	What is happening in the well	What the sensors show	Can a frequency change help?
Gas lock	Gas collects at the pump intake. The pump spins in gas and stops moving fluid.	Motor current dips and becomes unstable, intake pressure is erratic, vibration rises	Partly. We name the cause and flag inspection; no frequency model.
Underload	The pump pulls fluid faster than the reservoir can supply it. The fluid level over the pump falls.	Intake pressure falls, motor current sags	Yes. This is the one we give a frequency for.
High temperature	The motor is not cooled enough and heat builds up until its thermal limit trips.	Motor temperature climbs	Related, but not modelled. We flag inspection.
High discharge	Scale or a restriction downstream makes the pump push against rising pressure.	Discharge pressure climbs, current rises	Related, but not modelled. We flag inspection.

1.5 Two causes we removed, and why we kept their sensors

The project started with six causes. We removed two of them for field-engineering reasons:

- **Low voltage.** A voltage sag comes from the power grid, not from the well. Nothing in the well data can see it coming, and the drive's own undervoltage protection already handles it. Keeping it only added alarms. It was our worst cause at 19%.
- **Vibration.** A pump does not shut down because it vibrates. Vibration tells you something is wearing out. It is a symptom, not a trip cause.

We then tested removing the vibration and voltage **sensors** as well. That made things worse: gas lock identification dropped from a precision of 0.68 to 0.20. Gas at the intake makes the pump run unevenly, and that shows up in the vibration channel. So we dropped vibration as a **label** (something to predict) but kept it as a **signal** (something to predict from).

Presentation. If someone asks "why only four causes?", this is the answer: the two we removed cannot be predicted from well data, and the sensors that belonged to them are still used.

Part 2 · What we built

2.1 Three questions, answered in order

Stage	Question	Output
Stage 1 · Detect	Is this well going to trip in the next 30 minutes?	An alarm (yes / no)
Stage 2 · Cause	If yes, which of the four causes is developing?	A cause name
Stage 3 · Act	What should the operator change?	A frequency in Hz, or an inspection flag

2.2 The flow, from sensor to decision

```
SCADA sensors, one reading per minute
-> features (rolling 15 / 60 / 240-minute statistics + time since last trip)
-> Stage 1: XGBoost gives a trip-risk score for every minute
-> alarm policy: this well's top 5% of scores, held for 15 minutes
-> Stage 2: Random Forest names the developing cause
-> cause gate: underload -> frequency advice, clipped to the well's safe band
                other causes -> inspection prompt
-> live dashboard for the engineer
```

In parallel (not fed into the above): K-Means / DBSCAN map of operating regimes

The unsupervised branch runs on the side on purpose. It gives a map of how the wells operate, but we never let its output feed the supervised models, so the two can be judged separately.

2.3 What counts as success

- A warning only counts if it arrives **at least 30 minutes** before the trip. Less than that is too late to send anyone or change anything. That is detection, not prediction.
- The alarm load must be something a control room will keep trusting. If the system cries wolf all day, operators stop looking at it.
- Every result must be measured on wells the model never saw during training.

Part 3 · The data

3.1 Why the data is simulated

There is no public dataset with many ESP wells, one reading per minute, and labelled failure causes. So we built a simulator. It creates realistic sensor data for each well, including the slow build-up before each failure. This is the biggest limitation of the project and we say it openly: **the numbers show that the method works, not that it works on a real well**. The pipeline is built so that real well data can replace the simulated data without changing the model code.

3.2 Size

Item	Value
Wells	16
Time per well	6 months (182 days)
Resolution	1 reading per minute
Total rows	$16 \times 182 \times 1,440 = 4,193,280$ (about 4.19 million)
Training wells	13 (wells 1–7 and 11–16)
Held-out test wells	3 (wells 8, 9, 10), never used for training, tuning or choosing
Trips in the test wells	69

3.3 The well roster (wells_config.csv)

Every well is one row in a small file. The row says which causes that well suffers from, with a weight, and whether the well is used for training or testing. This file is the only thing you edit to add a well.

Well	Causes (weight)	Split
1, 5, 16	Gas lock (1.0)	train
2, 6, 11	Underload (1.0)	train
3, 12, 14	High temperature (1.0)	train
4, 13, 15	High discharge (1.0)	train
7	Gas lock 0.5 · High temperature 0.5	train
8	Underload 0.4 · High temperature 0.6	test
9	High discharge 0.5 · Gas lock 0.5	test
10	Gas lock 0.3 · High discharge 0.3 · Underload 0.4	test
17 (demo only)	Underload 0.5 · High discharge 0.5	test

Notice that every test well mixes two or three causes. That makes the test harder than the training wells, which mostly have one cause each. It is deliberate: real wells rarely fail for only one reason.

3.4 Equipment and the safe frequency band

Each well gets a motor, protector, gas separator and pump picked from a small catalogue with realistic ratings. From those ratings we work out the **safe frequency band**: the speed range that every part can handle.

safe minimum = the highest of the minimums (motor, pump)
safe maximum = the lowest of the maximums (motor, protector, pump)

Example: well 17. Motor HS 562 (35–75 Hz), protector rated to 72 Hz, pump GC-4000 (40–70 Hz, best efficiency at 55 Hz). Safe band = from $\max(35, 40) = 40$ Hz to $\min(75, 72, 70) = 70$ Hz. The pump normally runs at its best-efficiency point, 55 Hz. Any frequency we recommend is clipped to 40–70 Hz for this well. We never use one range for the whole field.

Wells that suffer from gas lock are given a weak gas separator (65% efficient) or none at all (25%). So the equipment explains the failure, as it would in a real field.

3.5 How a well lives inside the simulator

Every minute, each well is in one of four states:

State	What happens
Normal	The pump runs at its best-efficiency speed. Hidden "damage" counters slowly grow: gas build-up, heat load, scale (fouling), and the drop in fluid level (drawdown).
Onset	One counter has reached its limit, so a fault has started. Each minute the chance of a trip grows by 2 percentage points, so onset usually lasts only a few minutes.
Shutdown	The pump is off for 90 to 240 minutes. The counter that caused the trip is reset.
Ramp	The pump restarts just above its safe minimum and speeds up to normal over 5 to 10 hours.

The important detail is the **precursor**: part of each hidden counter leaks into the sensors before the fault officially starts. That is the early-warning signal the model learns to see. Part 8 explains why getting this physics right was the most important thing we did.

3.6 The sensor columns

Column	Unit	Meaning
frequency_hz	Hz	Pump speed set by the drive
motor_current_a	A	Motor current. Drops when the pump is starved or gas-locked
voltage_v	V	Supply voltage
Tm_motor_temp_f	°F	Motor temperature
Pi_intake_psi	psi	Pressure at the pump inlet. Tells you how much fluid is above the pump
Pd_discharge_psi	psi	Pressure at the pump outlet. Rises with restrictions downstream
vibration_g	g	How much the pump is shaking
Ti_intake_temp_f	°F	Fluid temperature at the intake. Recorded but not used by the model

The file also holds the labels: the state, a shutdown flag on the exact trip minute, the cause of that trip, the cause during onset minutes, the well's safe band, and a production estimate.

3.7 Keeping the test honest

- **Split by well.** Wells 8, 9 and 10 are locked away. The model never sees a single minute of them during training.
- **Split by time.** Inside the 13 training wells, the last 6 weeks are kept aside as **validation**. All tuning and all choices (which model, which alarm setting) are made on validation.
- **The test wells are used once**, at the end, to report the result. They never influence a decision.

Lesson. Twice during the project we picked a model by looking at the test wells. That is called leakage, and it makes results look better than they are. Both times we fixed the selection rule so it only looks at validation.

Part 4 · Features and labels

4.1 Rolling-window features

A single sensor reading is noisy. What matters is the trend. So for each of the 6 sensors we compute:

Feature	What it captures
15-minute mean	The recent level, smoothed
15-minute standard deviation	How jumpy the signal is right now
60-minute mean and standard deviation	The last hour's level and jumpiness
240-minute mean	The slow drift over 4 hours. This is where precursors show
5-minute change	How fast the signal is moving

That gives 6 sensors × 6 statistics = 36 rolling features, plus the 6 raw readings, the frequency, and one more feature below. **44 features in total.**

4.2 The run-life feature

Minutes since the last shutdown. In the field this is a standard monitoring signal (time since the last restart). In our data it turned out to be the single most important feature, because the hidden damage counters grow steadily between trips.

4.3 Per-well normalization

Every well sits at different absolute levels: different reservoir pressure, different supply voltage, different best-efficiency speed. A model that learns "intake pressure below 2,100 psi means trouble" is really learning which well it is looking at. For the root-cause model and the clustering, we therefore rescale each feature against **that well's own normal behaviour** (a z-score):

- Training wells: their normal behaviour is measured on the training period.
- Test wells and well 17: their normal behaviour is measured on **their first 30 days only**. That is the calibration window a new well would realistically have in the field.

4.4 Labels

- **Stage 1 label:** 1 if a trip happens in the next 30 minutes, otherwise 0. Only about **0.27%** of minutes are 1.
- **Stage 2 label:** the cause, but only on minutes where a fault is actually developing (onset minutes).

Lesson. An early version gave every minute in the 30 minutes before a trip the cause of that trip. But 28 minutes before a gas lock, the well still looks normal. The model was asked to name a cause from data that did not contain one, so it learned to always guess the most common cause and scored 39%. Training only on onset minutes fixed it (68.6%).

4.5 Handling the imbalance

With 0.27% positives, a model can be 99.7% accurate by always saying "no trip". To stop that, we keep every positive minute in training and only a sample of the negatives (40 negatives per positive), and we use class weights. The test is always measured on the full data, with nothing removed.

Part 5 · The models

5.1 Stage 1: three candidates

Model	In one sentence	Settings we used
Logistic Regression	Draws one straight boundary through the features. Simple and easy to explain.	Balanced class weights
Random Forest	Many decision trees, each trained on a random slice of the data, voting together.	150 trees, depth 9
XGBoost	Trees built one after another, each fixing the mistakes of the ones before.	150 trees, depth 5, learning rate 0.08

Each model outputs a **risk score** between 0 and 1 for every minute.

5.2 From a score to an alarm

A raw score is not an alarm. Two rules turn it into one:

- **Per-well threshold.** The alarm level is the top 5% of that well's own scores. A single fixed number does not work across wells: with the same model, this rule puts the alarm level at 0.003 for well 8 and at 0.14 for well 10, because the wells look so different.
- **Persistence.** The score must stay above the threshold for **15 minutes in a row**. A single noisy spike dies; a real fault, which builds over tens of minutes, survives.

Consecutive minutes above the line form one **alarm event**. If an alarm lasts 40 minutes, that is one interruption for the engineer, not 40.

5.3 When does an alarm catch a trip?

- The trip is **caught** if an alarm was active at any time in the **180 minutes** before it.
- The catch is **useful** if the alarm started at least **30 minutes** before the trip.
- An alarm that matches no trip is a **nuisance alarm** (a false alarm).

5.4 How the model and the alarm setting are chosen

We try every combination of persistence (1, 3, 5, 10, 15, 30 minutes) and threshold (top 5%, 2.5%, 1%, 0.5%, 0.1%) on the **validation** data, and score each one with a cost:

$$\text{cost} = (\text{missed useful trips} \times 200) + \text{nuisance alarms}$$

A miss is weighted 200 times a false alarm, because a missed trip costs hours of production and maybe the pump, while a false alarm costs an engineer a few minutes at a screen. The lowest cost wins.

Model	Best validation cost	Result
XGBoost	2,011	Selected
Random Forest	2,547	
Logistic Regression	5,721	

5.5 Stage 2: naming the cause

A Random Forest (200 trees, depth 10) looks at the normalized features plus three numbers from the equipment sheet: gas separator efficiency, number of pump stages, and motor horsepower. The equipment helps because gas lock and underload look similar on the sensors, but a well with a weak gas separator is far more likely to be gas-locked.

5.6 The unsupervised branch

- **K-Means (6 clusters)** groups minutes into operating regimes. Silhouette score 0.185 (the clusters overlap). Two of the six clusters hold almost all the minutes that come right before a trip.
- **DBSCAN** marks unusual minutes as outliers. Outliers have about twice the failure rate of normal minutes (0.23% against 0.11%).
- Useful as a map of how the wells behave. Too weak on its own to be a failure alarm.

Part 6 · How we measure

Four numbers carry almost all the meaning in this project, and they pull against each other.

Measure	Plain meaning	Good when
Catch rate	Of all the trips that really happened, how many did we warn about?	Higher
Lead time	How long before the trip did the warning come?	Longer; 30 min minimum
Nuisance alarms	Alarms with no trip behind them, per well per month	Lower
Root-cause accuracy	When we named a cause, how often was it right?	Higher

6.1 The standard ML words

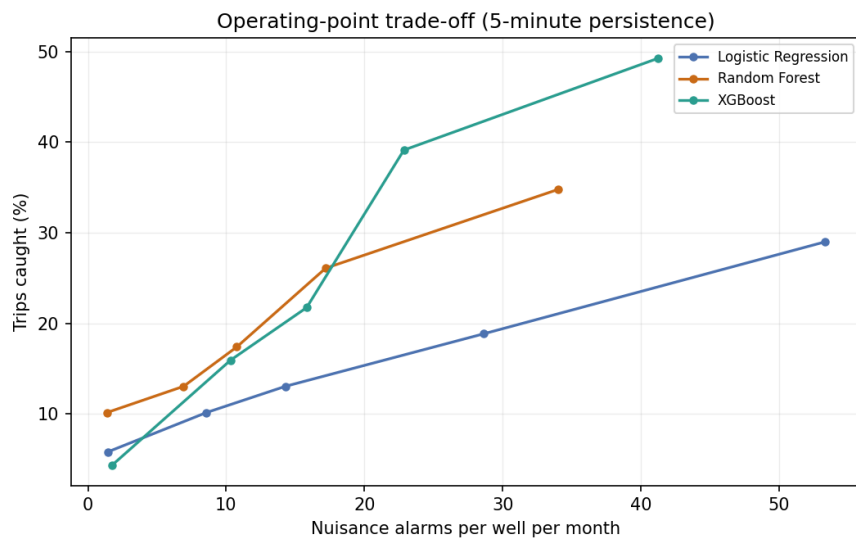
Word	Meaning	Example from our project
Precision	Of the times we said yes, how many were right	Gas lock precision 1.00: whenever we say gas lock, it is
Recall	Of the real cases, how many we found	Gas lock recall 0.55: we find about half of them
F1	One number that balances precision and recall	High discharge F1 = 0.88
Accuracy	Share of all minutes labelled correctly	Misleading here: 99.7% by always saying no
ROC-AUC	How well the scores rank risky minutes above safe ones (0.5 = coin flip, 1 = perfect)	XGBoost 0.768 on test wells

Remember. Accuracy is meaningless for rare events. A model that never raises an alarm is 99.7% accurate and completely useless. That is why we judge the models on events (trips caught, alarms raised) and not on minutes.

6.2 The trade-off nobody escapes

Make the system more sensitive and it catches more trips, but it also alarms more often when nothing is wrong. Measured on the test wells:

Setting	Catch rate	Alarms / well / month	What it feels like
Very cautious	3%	0.7	Almost never an alarm, almost never a warning
Cautious	35%	13.7	One alarm every two days
What we shipped	48%	18.7	One alarm every day and a half
Aggressive	51%	59.6	Two alarms a day for 3% more catches
Very aggressive	51%	128.6	Four alarms a day; nobody would keep trusting it



Each point is a different alarm setting. Up and to the left is better.

Part 7 · The results

Everything here is measured on wells 8, 9 and 10, which the models never saw.

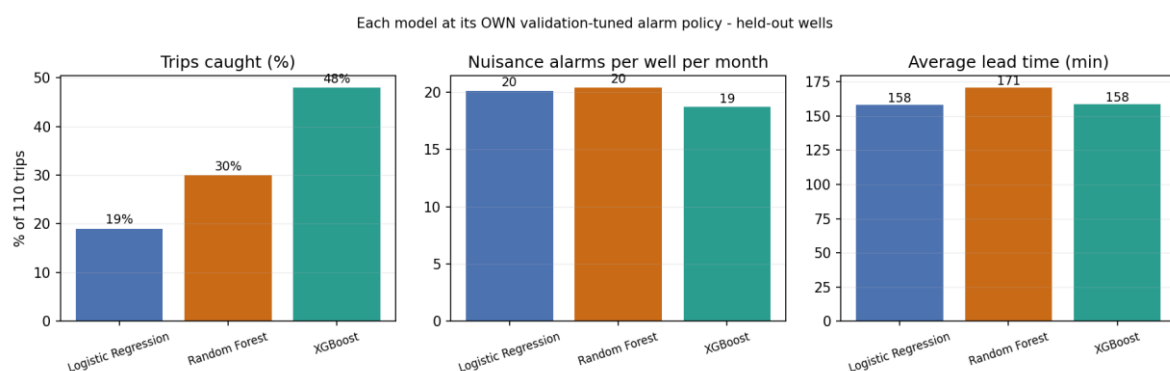
7.1 The headline

Measure	Result	In words
Trips caught	33 of 69 (48%)	We warned about roughly half the shutdowns
Caught with ≥ 30 min warning	31 of 69 (45%)	Almost every catch came early enough to act
Median warning	180 minutes	A typical warning came 3 hours ahead
Nuisance alarms	18.7 per well per month	About one every day and a half
Root-cause accuracy	68.6%	Right about two times in three, but it depends a lot on the cause
Downtime avoidable	about 99 hours	Across 3 wells in 6 months, if every warning were acted on (3 h per catch)

7.2 The three models side by side

Model	Caught	≥ 30 min	Alarms/well/ mo	Lead median (mean)	ROC-AUC val / test
Logistic Regression	13 (19%)	12 (17%)	20.1	180 (158) min	0.796 / 0.676
Random Forest	21 (30%)	20 (29%)	20.4	180 (171) min	0.937 / 0.736
XGBoost (selected)	33 (48%)	31 (45%)	18.7	180 (159) min	0.951 / 0.768

XGBoost catches more than twice what Logistic Regression does **and** raises fewer alarms. Winning on both at once is the sign of a genuinely better model, not a threshold moved to trade one for the other.



Each model at its own validation-tuned alarm setting.

7.3 The same models, measured per minute

These are the textbook metrics at a fixed 0.5 cut-off. They show why we do not use them to choose:

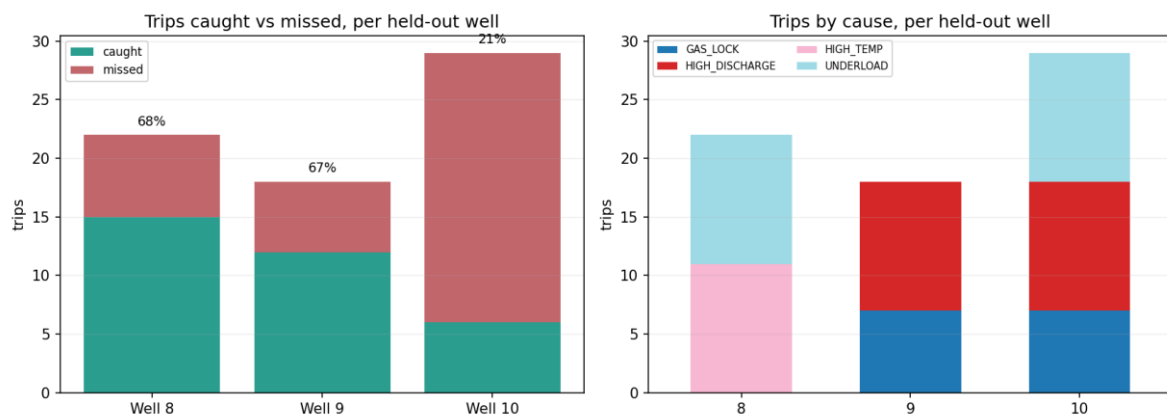
Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.944	0.010	0.204	0.019	0.676
Random Forest	0.981	0.020	0.125	0.034	0.736
XGBoost	0.993	0.019	0.036	0.025	0.768
Fixed-threshold SCADA rule	0.965	0.001	0.017	0.003	—
Persistence (last value)	0.9998	0.967	0.967	0.967	—

- Every model, and even the old SCADA rule, scores over 94% accuracy while catching almost nothing.
- Logistic Regression has the best recall here (0.204) but is the weakest at catching real trips.
- The persistence baseline looks perfect, but it copies the previous minute's label, which you only know once the trip is already less than 30 minutes away. It cannot warn anyone early.

7.4 Each test well

Well	What fails on it	Trips	Caught	Alarms/mo	Avg lead
8	Underload, high temperature	22	15 (68%)	20.3	171 min
9	High discharge, gas lock	18	12 (67%)	15.2	136 min
10	Gas lock, high discharge, underload	29	6 (21%)	20.6	174 min

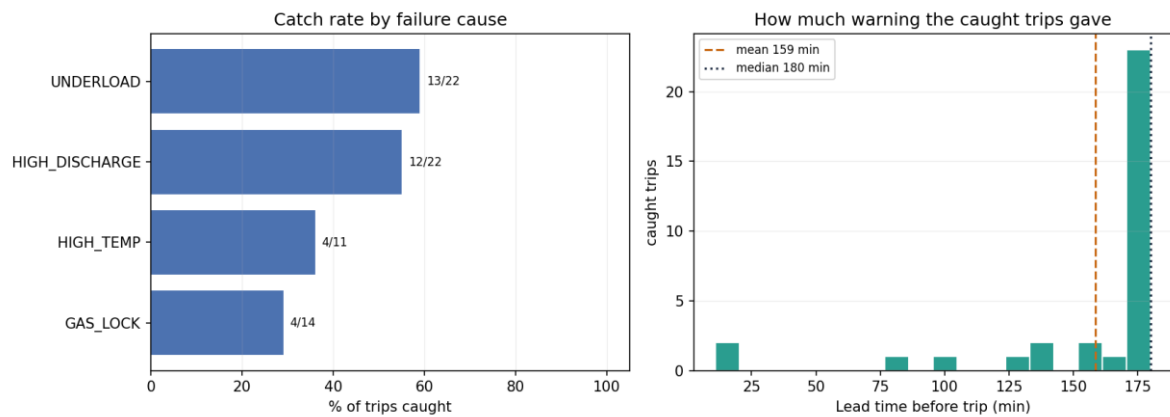
Wells 8 and 9 work well. Well 10 catches one trip in five. It has **three** failure modes at once, so its signals overlap, and because the alarm level is set from each well's own scores, a well that is rarely quiet ends up with a threshold that is too high. The method works best when a well has one or two distinct failure modes.



Caught and missed trips per test well (left) and the causes behind them (right).

7.5 Each cause

Cause	Caught	Catch rate
Underload	13 of 22	59%
High discharge	12 of 22	55%
High temperature	4 of 11	36%
Gas lock	4 of 14	29%



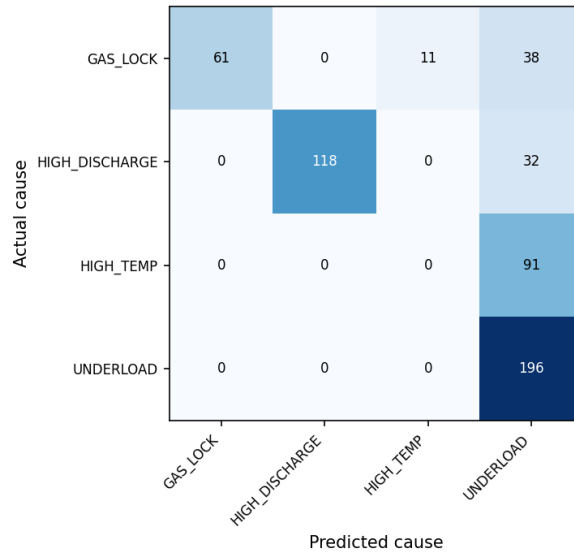
Catch rate by cause (left) and how much warning each caught trip gave (right).

7.6 Stage 2: naming the cause

Cause	Precision	Recall	F1	Onset minutes
Gas lock	1.00	0.55	0.71	110
High discharge	1.00	0.79	0.88	150
High temperature	0.00	0.00	0.00	91
Underload	0.55	1.00	0.71	196
Overall (accuracy 0.686)	0.64	0.59	0.58	547

- **Gas lock and high discharge are trustworthy.** When the model names one of them, it is always right.
- **High temperature fails completely.** The model never names it; all 91 of its minutes are called underload.
- That is also why underload looks weak: it catches every real underload, but half of what it calls underload is actually high temperature. Fixing high temperature would fix both.

Stage 2: Root-cause classification (held-out wells, normalized)

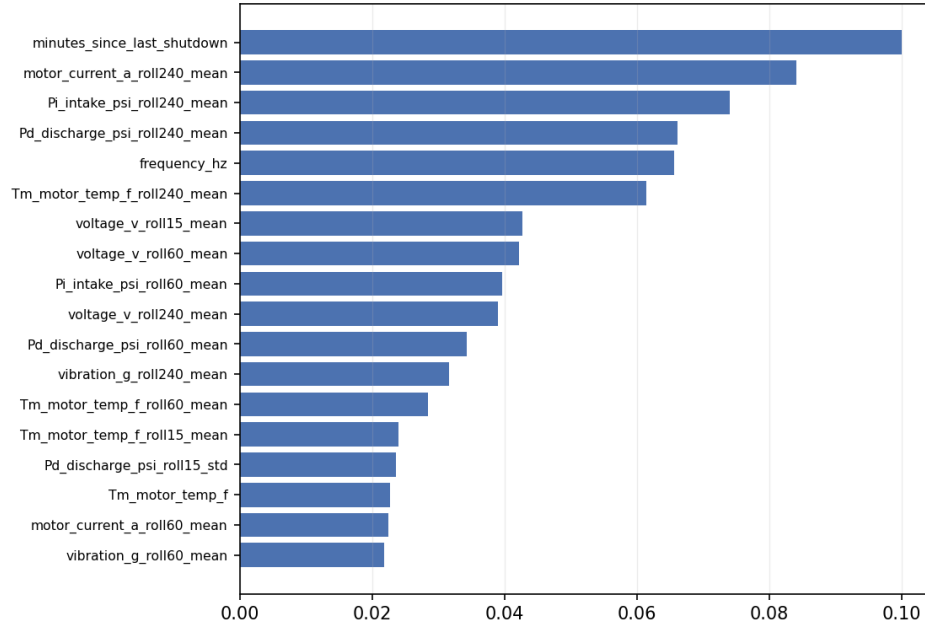


Rows are the true cause, columns the predicted one. The diagonal is what we want.

7.7 What drives the detector

The most important feature is the run-life feature (minutes since the last shutdown). Next come the 240-minute rolling means of current, intake pressure, discharge pressure and motor temperature. These slow-drift features are exactly where the precursors live.

What drives the detector - top 18 features (XGBoost)



The 18 most important XGBoost features.

Part 8 · How we got here

The final numbers are the end of nine experiments. Several of them made our headline number **worse**, and we kept the worse number because it was the honest one. Knowing this story is the best preparation for questions.

8.1 The experiments in order

#	What we tried	What happened	Decision
1	Start with six causes	32% caught, 61.6 alarms/well/month, cause accuracy 61.5%	Baseline
2	60-minute vs 30-minute horizon	60 minutes was worse: most of that hour still looks normal, so the labels fill with normal-looking data	Kept 30 min
3	Check why Stage 2 scored 39%	It predicted one cause for every row. Fixed by training on onset minutes only	Fixed: 66–69%
4	Count false alarms as events, not minutes	A "5.4% false alarm rate" was really about 208 alarms per well per month (7 a day)	Events from now on
5	Drop low voltage and vibration as causes	Catch rate 32% → 41%, cause accuracy 61.5% → 66.4%	Kept
6	Also drop their sensors	Gas lock precision collapsed 0.68 → 0.20	Rejected
7	Choose on event cost + 30-min rule instead of ROC-AUC	Catch rate 41% → 19%, but lead time 14 → 157 min and alarms 60.5 → 24.6	Kept
8	Measure signal against noise per cause	Found two physics errors in the simulator (see 8.2)	The big fix
9	Frequency control for underload	Works (22 → 0 trips) but costs 18.9% of production (Part 9)	Kept as a decision rule

8.2 The experiment that changed everything

For a long time detection was stuck at 19%. More features, other models, re-tuned thresholds: nothing moved it more than a point or two. Instead of trying yet another model, we asked a different question: **for each cause, how big is the warning signal compared with the sensor noise it has to be seen through?**

Cause	Signal vs noise	Catch rate at the time
Gas lock	6 σ (six times the noise)	45%
High temperature	3.1 σ	43%
High discharge	0.8 σ (smaller than the noise)	10%
Underload	0 σ (no warning signal at all)	6%

The catch rate followed the signal size almost exactly. **The problem was the data, not the model.** Two causes had been made impossible to predict by mistakes in the simulator:

- **Underload** fired as an instant random event. In reality, the fluid level falls gradually, which is exactly why pump-off controllers exist.

- **High discharge** had a 12 psi warning signal under 15 psi of sensor noise. That is mathematically undetectable.

	Before the fix	After the fix
Underload caught	6%	59%
High discharge caught	10%	55%
Overall caught	19%	48%
Alarms / well / month	24.6	18.7
Median lead time	157 min	180 min

Every number improved at the same time. Normally you buy a higher catch rate with more false alarms. When everything improves together, it means the signal was really there and had simply been buried.

The lesson to repeat in the presentation. Before spending another week on model architecture, measure whether what you ask the model to find is bigger than the noise it is hidden in. On real data, the same check tells you whether the sensors are good enough at all, in which case the fix is better instruments, not better software.

8.3 Ideas that did not work

Idea	Why it seemed good	What happened
Frequency-detrended features	Remove the normal effect of pump speed	Catch rate fell 49% → 28%
Relative-only features	Stop the model memorising which well it sees	Catch rate fell to 14%
F1-maximised thresholds	The textbook choice for imbalanced data	Picked a point with almost zero recall
Removing vibration & voltage sensors	They were no longer causes	Gas lock precision 0.68 → 0.20
Counting false alarms per minute	It is what the confusion matrix gives	Hid ~208 alarms/month as "5.4%"

Part 9 · Choosing the operating frequency

9.1 Why underload is special

Underload is the one failure an operator can fix by changing a setting. If the pump takes fluid faster than the reservoir gives it, you slow the pump down until they match. That is what a pump-off controller does. (An early version of our project wrongly listed underload as "not frequency-related". We corrected it.)

9.2 Step 1: learn the well from its own data (the regression)

For each well we fit a straight line between intake pressure and pump speed, using only that well's first 30 days of normal operation:

```
expected intake pressure = a - k × frequency
```

k is how many psi of intake pressure the well loses per extra Hz. No special well test is needed: the speed changes during restarts after trips give enough variation.

Well	k fitted from sensor data	True value in the simulator
8	9.27 psi/Hz	9.0
9	8.38 psi/Hz	9.0
10	10.36 psi/Hz	9.0
17 (dashboard)	9.93 psi/Hz	9.0

This is the most reusable piece of the project: it needs no labels and no failure history, only normal data.

9.3 Step 2: the recommendation

```
deficit = expected intake pressure - measured intake pressure
if deficit >= 18 psi:
    cut = (deficit + 12) / k          # the 12 psi is a safety margin
    new frequency = current frequency - cut, clipped to the safe band
    if even the safe minimum cannot recover it -> escalate to inspection
```

Example from the dashboard, well 17: running at 55 Hz, intake pressure 26 psi below expected, $k = 9.93$. $\text{Cut} = (26 + 12) / 9.93 \approx 3.8$ Hz, so the advice is about **51 Hz**, inside the 40–70 Hz band.

9.4 Step 3: does it work?

We re-ran the same wells with the same random seeds, once with the control off and once with it on, so the only difference is the control itself.

	Control off	Control on
Underload trips (wells 8 and 10)	22	0
Total downtime	140 h	69 h
Production	baseline	–18.9%

The control works perfectly and is still usually the wrong thing to do: we gave up 18.9% of the oil to recover a small amount of uptime. **Avoiding trips was never the goal. Producing oil is.** Running slower loses oil every minute; running too fast loses whole hours to trips. The best answer is in between.

9.5 The part that transfers to a real field: the break-even rule

Well	Setpoint	% of best-efficiency speed	Trips avoided	Production given up	Break-even downtime per trip
8	46.8 Hz	90%	11	-9.2%	36.4 h
8	49.4 Hz	95%	5	-4.6%	39.8 h
10	48.6 Hz	90%	12	-9.0%	32.9 h
10	51.3 Hz	95%	8	-4.4%	24.1 h

```

if (hours a trip keeps this well down) > (break-even hours, about 33 to 36 h):
    slow the well down
else:
    accept the trips and keep the production

```

A well that restarts automatically in a few hours: accept the trips. A remote well that waits a day or two for a crew: slow it down. **Same well, opposite answer, decided by restart time**, which is a field fact, not a modelling choice. The dashboard shows this rule next to every frequency advice (35 h for well 17).

9.6 One result we deliberately ignore

A frequency sweep in the simulator says production keeps rising up to 120% of the best-efficiency speed (+17%), even though trips triple. We do not trust it: the simulator lets flow rise with speed without limit, does not model pump submergence (you cannot pull the fluid below the pump), and gives a trip no cost in pump life. Field practice is not to run above the best-efficiency point, so our advice never goes above it.

Part 10 · The live dashboard and the demo

10.1 What it is

A single web page, [6_dashboard/ESP_Live_Dashboard.html](#), that replays a brand-new well as if its sensor data were arriving live, minute by minute, at any speed. It opens in any browser and needs no internet. Step 8 of the main notebook builds it.

10.2 The rules that keep it honest

- **Well 17 did not exist during training.** It is created after the models are saved.
- **The models are frozen.** They are loaded from disk and only predict. Nothing is re-trained.
- **Days 1 to 30 are calibration.** The well's normal levels and its alarm threshold come from these 30 days only. No labels are used, and no future data.
- **The true cause is revealed only when the pump trips**, exactly as an engineer would find out.

10.3 A tour of the screen

Area	What it shows
Top bar	CALIBRATING / LIVE / PAUSED, the simulated date and time, and the controls (Guided demo, Play, speed, Next alarm, Restart)
Six cards	Pump running or tripped · trip risk for the next 30 minutes · alarm status and suspected cause · shutdowns warned so far · median warning · nuisance alarms
Big chart	Risk score (blue), alarm threshold (dashed amber), alarm periods (shaded), real trips (red lines), and an arrow showing the minutes of warning before each caught trip
Six small charts	The raw sensors: current, intake and discharge pressure, motor temperature, vibration, frequency (with the advised frequency as a dashed green line)
Right side	Root-cause model, frequency advice with the break-even rule, the event log, and the well's equipment
Bottom strip	All six months: green marks are warned trips, red marks are missed ones. Click to jump



Well 17 during an underload alarm: risk far above the threshold, cause named, and frequency advice of 55.0 → 51.1 Hz.

10.4 Well 17 results

Measure	Well 17 (dashboard)
Trips warned (after the 30-day calibration)	9 of 17 (53%), all at least 30 minutes ahead
Median warning	180 minutes
Cause correct at the moment of each warned trip	9 of 9
Cause correct on onset minutes	116 of 117 (99%)
Nuisance alarms	about 37 per month
Drawdown coefficient k	9.93 psi/Hz (true 9.0)
Frequency advice during underload alarms	55 → about 51 Hz, worth it only if a trip costs more than 35 h

All the underload trips were warned; all the high-discharge trips were missed.

10.5 Why these numbers are not the same as the report

- **It is a different well.** The report measures wells 8, 9 and 10. Well 17 has no high temperature, which is the cause the model fails on, so its cause accuracy is higher.
- **The threshold comes from the first 30 days only.** A live system cannot see the future, so it cannot set its threshold from all six months as the offline evaluation does. This gives more nuisance alarms (37 against 18.7).
- **The 4 trips during calibration are not counted,** so it is 17 trips, not 21.

10.6 The demo video, second by second (57 seconds)

Time	What happens on screen
0–5 s	CALIBRATING: the first 30 days fly past while the model learns the well's normal levels
5–10 s	LIVE: data starts streaming at high speed
10–25 s	Slows down: risk climbs, ⚠ ALARM fires, the model names UNDERLOAD, frequency advice appears
about 24 s	The pump trips. The log says: warned 180 min ahead, cause correct
25–50 s	Speeds through the rest of the six months, counters climbing
end	Summary: 9 / 17 warned, 180 min median warning, 99% cause, 37 nuisance alarms per month

Part 11 · Presentation prep

11.1 The slides and the one message of each

Slide	Title	The one thing to say
1	Title	Predicting ESP shutdowns hours ahead, naming the cause, and advising a safe frequency
2	Agenda	Problem, data and AI, then the platform and a live demo
3	Problem statement	Most failures build up over hours; existing protection only reacts
4	Project objective	Three questions; a warning counts only if it comes 30 minutes early
5	Data preparation	16 simulated wells, 4.19M minutes; wells 8, 9, 10 locked away
6	Modeling workflow	Detect, name the cause, advise a frequency; chosen on validation cost
7	Model evaluation	48% caught, 3 hours ahead, 18.7 alarms per well per month, 68.6% cause accuracy
8	Platform	Risk, cause, frequency and when not to use it; about 99 hours of avoidable downtime
9	Live demo	A well the model never saw, streamed live; nothing is re-trained

11.2 Questions we are likely to get

Why is the data simulated?

No public dataset has many ESP wells at one-minute resolution with labelled failure causes. The pipeline reads a standard file format, so real well data can replace the simulation without changing the model code.

Only 48%? Why not higher?

We could reach 51%, but it costs three to seven times the alarms. An operator who stops trusting the alarms has a worse system. 48% with warnings three hours ahead is the operating point a control room can live with.

Why XGBoost?

It had the lowest cost on the validation data, and on the test wells it caught the most trips and raised the fewest alarms at the same time. The test wells confirmed the choice; they did not make it.

Why not report accuracy?

Only 0.27% of minutes come before a trip. A model that never raises an alarm is 99.7% accurate and useless.

How do you know the model is not cheating?

Wells 8, 9 and 10 were never used for training, tuning or selection. Twice we caught ourselves choosing on test wells and fixed the rule. And the live demo uses well 17, created after the models were saved.

Why does high temperature fail?

Our shared rolling features do not capture its heat build-up shape, so its minutes look like underload. Dedicated thermal-trend features are our first next step.

Why is well 10 so poor (21%)?

It has three failure modes at once. The signals overlap, and its alarm threshold ends up too high.

What is k?

How many psi of intake pressure a well loses per extra Hz. We fit it from each well's own first 30 days with a straight-line regression. It came out at 8.4 to 10.4 against a true 9.0.

If the frequency control removes all underload trips, why not always use it?

It cost 18.9% of production. It only pays if a trip would keep the well down longer than about 35 hours, for example a remote well waiting for a crew.

Why does the dashboard show 37 nuisance alarms when the report says 18.7?

Different well, and the live system sets its threshold from the first 30 days only, because it cannot see the future. The offline evaluation used the whole period.

What was the most important thing you learned?

Measure the signal against the noise before changing the model. Our catch rate was stuck at 19% until we found two physics errors in the data; fixing them took it to 48% and improved every other number too.

What would you do with more time?

Features for high temperature, test on real historical wells, separate alarm thresholds per cause, and add pump submergence limits to the simulator.

Part 12 · The project folder and how to run it

12.1 What is where

Folder / file	What it holds
1_run_in_colab/ESP_Capstone_Colab.ipynb	The main notebook: the whole project in one file, Steps 0 to 8
ESP_NewWell_Demo_Colab.ipynb / _VSCode.ipynb	Older demo notebooks that score well 17 in a notebook instead of the dashboard
2_code/	The same code as separate scripts, with wells_config.csv and HOW_TO_ADD_A_WELL.md
3_results/	Saved results: results.txt, frequency results, equipment specs, and earlier configurations
4_charts/	Every chart as PNG, plus the dashboard screenshot
5_documents/	Report (Word), presentation (with the demo video), summary PDF, action plan
6_dashboard/	build_dashboard.py, the page template, and ESP_Live_Dashboard.html

12.2 The notebook, step by step

Step	What it does	Time
0	Install XGBoost, create the working folder	seconds
1	Write the 16-well roster	seconds
2	Equipment and safe frequency band for each well	seconds
3	Simulate the SCADA data (4.19M rows); deletes any old data file first	about 5 min
4	Full analysis: clustering, Stage 1, Stage 2, alarm policy, all charts, results.txt	about 6 min
5	Frequency study: k, sweep, break-even	about 4 min
6	Print results and show charts	seconds
7	Save the trained models to model_store/esp_models.joblib	seconds
8	Build the live dashboard for well 17 and open it	about 30 s

12.3 Running it

- **Google Colab:** upload the notebook, choose a CPU runtime, run all cells in order. Nothing to download. (Step 8 needs the 6_dashboard folder, so it only works when run from the project folder.)
- **VS Code:** open the notebook, choose the **Python 3.12** kernel, run all cells. Files go to C:\content\esp_multiwell.
- **Just the dashboard:** double-click 6_dashboard/ESP_Live_Dashboard.html. No code needed. Press **Guided demo**.

12.4 Things that can trip you up

- **Library versions change the Stage 1 numbers a little.** On a laptop with newer XGBoost and scikit-learn, the same code caught 29 of 69 trips (42%) instead of 33 (48%). The data, Stage 2 and all frequency results came out exactly the same. The report numbers come from the Colab run.
- **An old data file gets reused.** The data generator only adds wells that are missing. Step 3 now deletes the old file first; if you run the scripts by hand, delete `multiwell_scada_10wells_6mo.csv` yourself.
- **Use the same Python for saving and loading models.** The saved models load only in the Python version and libraries that created them (3.12 on our machine).

Part 13 · Limitations and next steps

13.1 What we are confident about

- The pipeline works end to end and reproduces from scratch.
- The results are not inflated by leakage: three wells were never touched.
- The model learns physics, not noise: when two causes got realistic warning signals, their catch rates rose to match.
- Gas lock and high discharge are named with 1.00 precision on unseen wells.
- The break-even rule is built only from measured quantities, so it carries over to real economics.

13.2 What we are not sure of

- **The data is simulated.** This is the biggest caveat and applies to every number.
- **52% of trips are still missed.** The system adds to the existing protection; it does not replace it.
- **High temperature fails completely** in Stage 2.
- **Unseen wells are harder:** ROC-AUC 0.951 on validation against 0.768 on the test wells; well 10 reaches only 21%.
- **The production economics are invented;** only the break-even rule survives outside the simulator.

13.3 Next steps, in order

1. **Features for high temperature,** the one class that is broken rather than weak.
2. **Run on real historical wells,** starting with the signal-versus-noise check from Part 8.
3. **Separate alarm thresholds per cause.** Gas lock is at 29% while underload reaches 59% under one shared setting.
4. **Add submergence and pump-curve limits** to the simulator, so the frequency sweep gives an answer worth acting on.

Glossary

Term	Meaning
Alarm event	A run of consecutive minutes above the alarm threshold. Counted once, however long it lasts.
Alarm policy	The rule that turns risk scores into alarms: top 5% of the well's own scores, held for 15 minutes.
BEP (best-efficiency point)	The pump speed at which it works most efficiently. The normal operating speed.
Break-even downtime	How long a trip must keep a well down before slowing the well is worth the lost oil.
Calibration window	A new well's first 30 days, used to learn its normal levels. No labels needed.
Catch rate	Share of real trips that had an alarm in the 180 minutes before them.
Class imbalance	When one class (trips) is far rarer than the other. Here 0.27% of minutes.
Confusion matrix	A table of true cause against predicted cause.
DBSCAN	Clustering that also marks unusual points as outliers.
Drawdown coefficient (k)	psi of intake pressure lost per extra Hz of pump speed.
ESP	Electrical submersible pump: the pump at the bottom of the well.
Feature	A number computed from the sensors that the model uses as input.
Frozen model	A trained model saved to disk and only used to predict, never re-trained.
Held-out wells	Wells 8, 9 and 10: kept away from training and used only for the final test.
K-Means	Clustering that splits data into k groups (we used 6).
Lead time	Minutes between the start of a useful alarm and the trip.
Leakage	Letting test data influence a decision, which makes results look better than they are.
Nuisance alarm	An alarm with no trip behind it (a false alarm).
Onset	The minutes when a fault has started but the pump has not tripped yet.
Persistence	How many minutes in a row the score must stay above the threshold before an alarm fires.
Precursor	The early-warning sign in the sensors before a fault officially starts.
Protector	The ESP section that seals the motor and carries the pump's thrust.
psi	Pounds per square inch, the unit of pressure.
Run-life feature	Minutes since the well's last shutdown. Our most important feature.
Safe frequency band	The speed range every part of a well's ESP can handle. Different for each well.
SCADA	The system that collects the sensor readings from the wells, here once per minute.
Silhouette score	How well separated clusters are, from -1 to 1. Ours is 0.185 (overlapping).
Trip	An automatic shutdown of the pump by its protection.
Validation	Data kept aside from training (the last 6 weeks of the training wells) for tuning and choosing.
Workover	Pulling the ESP out of the well to repair or replace it. Very expensive.
XGBoost	Gradient-boosted decision trees: our selected Stage 1 model.

Term	Meaning
z-score normalization	Rescaling a value by subtracting the well's normal mean and dividing by its spread.